MUSICGROWTH LAB

AI-Powered Strategic Positioning for Independent Artists

TEAM EXECUTION BLUEPRINT

Complete A-to-Z Workflow | 6-Member Team | 17-Day Sprint

PES University — Centre for Innovation & Entrepreneurship | April 9, 2026

Part 1: Brutally Honest Assessment of the Idea

Before writing a single line of code, every team member must understand this section completely. This is not optional reading.

1.1 What's Actually Strong

Genuine Strengths Worth Defending

The strategic positioning reframe (from 'recipe' to 'navigation system') is genuinely sophisticated and judges will respect it.

The 3-path model (Mainstream / Niche / Hybrid) maps cleanly onto 3 separate ML models — this is defensible, explainable, and technically sound.

Sound DNA + cosine similarity + clustering is a legitimate ML stack that shows technical depth.

The market gap angle ('underserved feature combinations') is novel and will impress if executed well.

SHAP explainability gives you a strong answer to 'Why should I trust this?' from judges.

1.2 Weaknesses — Be Brutally Honest

WEAKNESS	WHY IT'S A PROBLEM	HOW TO MITIGATE
Predicting virality from audio features alone	R^2 will likely be 0.2–0.4. Audio features ≠ success. Label backing, timing, playlists matter enormously. Judges will call this out.	Reframe as 'conditional positioning scenarios', NOT 'your song will get X streams.' Never promise hard numbers.
'Add 20% tempo → grow 40%' is scientifically dubious	This is a correlation claim dressed as causation. One	Drop this framing entirely. Use: 'Songs with these feature profiles historically appeared in

	judge who knows music industry will demolish you.	X% more playlists.' Conditional, not causal.
Dataset availability for real trends	You do NOT have timestamped Spotify trend data. You'll simulate it. If a judge asks 'is this real trend data?' you need an honest answer.	Use Kaggle Spotify datasets (~600K tracks). Be transparent: 'We model feature-based positioning clusters, not live streaming trends.'
Scope creep in 17 days	6 people building ML + UI + integration + presentation in 17 days is tight. If ML takes longer, you'll have a broken demo.	MVP first. Lock core features by Day 10. Days 11-17 are polish and demo prep. No new features after Day 10.

1.3 Improvements That Will Make You Win

- Add SHAP values to every recommendation — instead of 'increase energy', say 'energy contributes 34% to playlist placement probability in this genre cluster.' Judges love explainability.
- Build 3 separate classifiers for the 3 paths instead of one generic growth predictor — this directly maps the strategic framework to ML architecture.
- Use a real dataset: Kaggle's 'Spotify Tracks Dataset' has ~600K tracks with audio features + popularity. This is free and sufficient.
- Add a live upload demo: user uploads an MP3 → librosa extracts features → system positions them in real time. This is the money shot for the demo.
- Market Gap Visualization: a scatter plot of genre saturation (X-axis) vs. demand proxy (Y-axis, using popularity of similar artists) is highly visual and unique.
- Honest confidence intervals: instead of 'you will grow 40%', show a range with uncertainty. This actually looks MORE sophisticated to judges.

Part 2: Team Roles and Responsibilities

The team is divided into 6 specialized roles. Each role has a primary responsibility and supports other roles during integration. Read your full role section carefully.

#	ROLE TITLE	PRIMARY RESPONSIBILITY	KEY DELIVERABLE
M1	Data Engineer	Dataset acquisition, cleaning, feature engineering, Sound DNA vectors	Clean dataset + feature matrix + embeddings (CSV/Parquet)
M2	ML Engineer — Clustering & Similarity	Sound DNA embedding, PCA/UMAP, K-Means clustering, cosine similarity engine	Similarity engine module + cluster labels + 2D embedding visualization
M3	ML Engineer — Growth Prediction & SHAP	3-path growth models (XGBoost), SHAP explainability, feature importance	3 trained models + SHAP plots + feature importance report
M4	ML Engineer — Market Analysis & Trend	Genre saturation analysis, market gap detection, trend proxy modeling, rule-based recommendation layer	Market gap scatter data + trend visualization + recommendation engine
M5	Frontend & UX Developer	Streamlit dashboard, all visualization screens, upload interface, radar charts, growth simulator UI	Fully functional Streamlit app with all 5 screens
M6	Integration Lead & Pitch Captain	Backend pipeline wiring, model evaluation metrics, deployment (local), presentation deck, pitch practice	Working end-to-end pipeline + evaluation report + polished pitch deck

Part 3: Detailed Role Breakdowns — Step by Step

MEMBER 1 — Data Engineer

MISSION: Build the foundation everything else depends on. If your data is bad, all 5 other members fail.

Days 1–4. Critical dependency for M2, M3, M4.

Step 1: Dataset Acquisition (Day 1)

- Download 'Spotify Tracks Dataset' from Kaggle (search: 'spotify dataset audio features kaggle'). The recommended one has ~600K tracks with: track_id, name, artists, album_name, track_genre, popularity, duration_ms, explicit, danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, time_signature.
- Also download 'Spotify Artist Data' dataset for follower counts (needed to filter 1M+ follower artists for reference matching).
- Store raw files in /data/raw/. Never overwrite raw data — all outputs go to /data/processed/.
  - If Kaggle is unavailable, use the Spotify Web API with spotipy library: pip install spotipy. Pull audio features for ~10,000 songs across genres.

Step 2: Exploratory Data Analysis (Day 1–2)

- Run a Jupyter notebook: EDA.ipynb. Check null values, data types, distributions of each feature.
- Plot histograms for: tempo, energy, danceability, valence, acousticness, loudness, popularity. Send screenshots to the team group chat. M3 and M4 need to understand the distributions before building models.
- Check class imbalance in genres — some genres may have 10x more songs than others. Document this.
- Identify outliers: songs with popularity=0 may be inactive tracks — decide whether to drop them (recommended: drop popularity<5 for modeling).

Step 3: Feature Engineering & Normalization (Day 2–3)

- Normalize ALL numerical features using StandardScaler from sklearn. Save the scaler object using pickle — M5 needs this to scale user-uploaded song features at inference time.
  - from sklearn.preprocessing import StandardScaler; scaler = StandardScaler(); X_scaled = scaler.fit_transform(X)
- Create 'popularity buckets' for classification: Low (0–30), Medium (31–60), High (61–100). This is the target variable for M3's classifiers.
- Create a binary 'mainstream_ready' flag: popularity > 60 AND in top-5 most common genres. This is for Path A (Mainstream) model.
- Create a 'niche_success' flag: popularity > 40 AND genre is NOT in top-5 most common. This is for Path B (Niche) model.
- Create the Sound DNA feature vector: a 10-dimensional vector [danceability, energy, loudness_normalized, speechiness, acousticness, instrumentalness, liveness, valence, tempo_normalized, explicit_binary]. This is the core representation.
- Export final processed dataset as: /data/processed/spotify_processed.csv and /data/processed/sound_dna_matrix.npy

Step 4: Reference Artist Dataset (Day 3–4)

- Filter artists with follower count >= 1M (use the artist dataset). Extract their average audio feature profiles per artist.
- Create /data/processed/reference_artists.csv with columns: artist_name, genre, avg_danceability, avg_energy, avg_valence, avg_tempo, avg_acousticness, follower_count, popularity_avg.
- This file is directly consumed by M2's similarity engine.

Deliverables Checklist for M1

- /data/processed/spotify_processed.csv (cleaned, normalized)
- /data/raw/ (original untouched data)
- /data/processed/sound_dna_matrix.npy (feature vectors)
- /data/processed/reference_artists.csv
- /models/scaler.pkl (fitted StandardScaler)
- EDA.ipynb with key findings documented
- README in /data/ explaining every column

MEMBER 2 — ML Engineer: Clustering & Similarity

MISSION: Build the 'where am I in music space' engine. This is the visual heart of the product — the map that tells artists where they stand.

Days 3–7. Depends on M1's deliverables.

Step 1: Dimensionality Reduction for Visualization (Day 3–4)

- Load sound_dna_matrix.npy from M1. Apply UMAP first (better than PCA for music data — preserves local structure): pip install umap-learn
  - import umap; reducer = umap.UMAP(n_components=2, random_state=42); embedding_2d = reducer.fit_transform(X_scaled)
- Also run PCA for comparison: from sklearn.decomposition import PCA. Use PCA for the model input (keeps explainability), UMAP for visualization only.
- Save both: /models/umap_reducer.pkl and /models/pca_reducer.pkl. Save 2D coordinates as /data/processed/embedding_2d.csv with columns: track_id, umap_x, umap_y, genre, popularity.

Step 2: Clustering (Day 4–5)

- Run K-Means on the PCA-reduced features (not UMAP — K-Means works better in lower dimensions from PCA). Try k=10 to k=20. Use silhouette score to pick optimal k.
  - from sklearn.metrics import silhouette_score — plot scores for k=8 to k=25, pick the elbow.
- Label each cluster with its dominant genre using mode of genre column within each cluster. Name clusters meaningfully: e.g., 'Energetic Dance', 'Melancholic Indie', 'Acoustic Folk'.
- Also run DBSCAN as a secondary clustering for detecting outliers (unusual/experimental artists who don't fit any cluster). Mark these as 'UNCHARTED TERRITORY' — this is a positive differentiator in the UI.
- Save cluster labels as /data/processed/cluster_labels.csv. Export the K-Means model as /models/kmeans.pkl.

Step 3: Cosine Similarity Engine (Day 5–6)

- This is the core function called at inference: user uploads a song, gets feature vector, finds most similar reference artists.
- Build similarity.py module with function: get_similar_artists(user_vector, reference_df, top_n=5) -> returns sorted dataframe with similarity scores.
  - from sklearn.metrics.pairwise import cosine_similarity — compute similarity between user's Sound DNA vector and each reference artist's average vector.
- Return format: [{artist: 'The Japanese House', similarity: 0.81, genre: 'bedroom pop', followers: 2.1M}, ...]. This feeds directly into M5's UI.
- Critical: also return the DIFFERENCES not just similarities. For each dimension, compute delta = user_value - reference_artist_value. Label each delta as STRENGTH (positive, intentional) or GAP (negative, potential improvement).

Step 4: Genre Space Visualization Data (Day 6–7)

- Produce the UMAP scatter plot data with: cluster color, genre label, popularity heatmap overlay, and a marker for where the user's uploaded song sits.
- Export as /data/processed/genre_space_viz.json — M5 uses this to build the Plotly scatter chart.
- Also create cluster_profiles.json: for each cluster, the centroid feature values + dominant genres + average popularity + artist examples.

Deliverables Checklist for M2

- /models/umap_reducer.pkl and /models/pca_reducer.pkl
- /models/kmeans.pkl
- /data/processed/embedding_2d.csv
- /data/processed/cluster_labels.csv
- /data/processed/cluster_profiles.json
- /data/processed/genre_space_viz.json
- src/similarity.py module with get_similar_artists() function
- clustering_notebook.ipynb with silhouette score analysis

MEMBER 3 — ML Engineer: Growth Prediction & SHAP

MISSION: Build the 3-path prediction engine with SHAP explanations. This is the most technically defensible part of your project. Do NOT oversell predictions — undersell and overdeliver on explainability.

Days 4–9. Depends on M1's processed data and feature flags.

Step 1: Understand the 3 Models You Need to Build

MODEL	TARGET VARIABLE	TRAINING DATA FILTER
Path A: Mainstream	mainstream_ready flag (binary classifier)	All songs, predicting likelihood of mainstream success
Path B: Niche	niche_success flag (binary classifier)	Non-top-5-genre songs only, predicting niche audience success
Path C: Hybrid	popularity_bucket (3-class: Low/Med/High)	All songs with cluster labels from M2 as additional feature

Step 2: Build Path A — Mainstream Classifier (Day 4–5)

- Algorithm: XGBoost classifier (pip install xgboost). Why XGBoost: handles mixed data well, fast, works with SHAP out of the box.
- Features: the 10-dim Sound DNA vector + cluster_label (from M2) + genre_encoded (label encode genre).
- Train/test split: 80/20, stratified on target. Use cross-validation (5-fold) to report stable accuracy.
- Tune hyperparameters using GridSearchCV or Optuna on: n_estimators, max_depth, learning_rate.
- Save model: /models/path_a_mainstream.pkl. Log: accuracy, precision, recall, F1, AUC-ROC.
- CRITICAL: Also compute calibrated probabilities (CalibratedClassifierCV) — you'll show 'Path A probability: 73%' in the UI.

Step 3: Build Path B & C (Day 6–7)

- Follow identical process for Path B (niche_success) and Path C (popularity_bucket multiclass).
- For Path C, use macro-averaged F1 as the metric (handles class imbalance better than accuracy).
- All models saved as: /models/path_b_niche.pkl and /models/path_c_hybrid.pkl.

Step 4: SHAP Explainability (Day 7–8) — DO NOT SKIP THIS

- Install: pip install shap. This is what separates you from every other team.
- For each model, compute SHAP values for the test set and for individual predictions.
  - import shap; explainer = shap.TreeExplainer(model); shap_values = explainer.shap_values(X_test)
- Generate 3 types of SHAP plots and save them as PNG files: (1) Global feature importance bar chart — 'energy contributes most to mainstream success'. (2) Beeswarm plot — distribution of feature effects. (3) Waterfall plot for individual prediction — 'for YOUR song, energy pushed probability up by +12%'.
- Write a function: explain_prediction(user_vector, model, explainer) -> returns top 3 positive and top 3 negative feature contributions as text. M5 uses this in the UI.
- Save SHAP explainer objects: /models/shap_explainer_a.pkl, /models/shap_explainer_b.pkl, /models/shap_explainer_c.pkl.

Step 5: Model Evaluation Report (Day 8–9)

- Create evaluation.ipynb documenting: confusion matrices for all 3 models, classification reports (precision/recall/F1 per class), ROC curves, SHAP global importance plots, comparison of all 3 models side by side.

- IMPORTANT: Write a 1-paragraph 'limitations' section. Honest acknowledgment of model limits impresses judges MORE than fake confidence. Example: 'These models capture audio-based positioning patterns but do not account for marketing, label support, or social media amplification, which are significant drivers of streaming success.'

Deliverables Checklist for M3

- /models/path_a_mainstream.pkl, path_b_niche.pkl, path_c_hybrid.pkl
- /models/shap_explainer_a.pkl, shap_explainer_b.pkl, shap_explainer_c.pkl
- src/predictor.py — predict_paths(user_vector) -> returns {path_a_prob, path_b_prob, path_c_class, shap_explanations}
- /reports/shap_global_a.png, shap_global_b.png, shap_global_c.png
- evaluation.ipynb with all metrics

MEMBER 4 — ML Engineer: Market Analysis & Recommendations

MISSION: Build the 'market intelligence' layer. Your outputs are what makes MusicGrowth Lab feel like a real strategic tool rather than just another ML demo.

Days 4–9. Partially depends on M2's cluster labels.

Step 1: Genre Saturation Analysis (Day 4–5)

- Goal: identify which genre clusters are overcrowded vs. which have high demand but few artists.
- Saturation metric: count of songs per cluster / total songs. High count = saturated market.
- Demand proxy: average popularity of songs within each cluster. High popularity + low count = MARKET GAP.
- Create market_gap_df with columns: cluster_id, cluster_name, song_count, avg_popularity, saturation_score, demand_score, opportunity_score (demand/saturation).
- Save as /data/processed/market_gaps.csv. This powers the market gap scatter plot in M5's UI.

Step 2: Feature Combination Analysis (Day 5–6)

- Identify 'underserved feature combinations': pairs of features (e.g., high energy + high acousticness) that have high popularity but few songs.
  - Use a 2D binning approach: divide each feature into 5 quantile bins, count songs per bin-combination, compute avg popularity per bin. Low count + high avg popularity = opportunity zone.
- Save top 10 opportunity zones as /data/processed/feature_opportunities.json. Format: [{feature_1: 'energy', range_1: '0.7-0.9', feature_2: 'acousticness', range_2: '0.3-0.5', avg_popularity: 67, song_count: 142, opportunity_score: 0.87}, ...].

Step 3: Recommendation Engine (Day 6–8)

- This is a HYBRID system: rule-based filtering + ML ranking.
- Function: generate_recommendations(user_vector, predicted_paths, similar_artists, market_gaps) -> returns 3 path-specific recommendation sets.
- For Path A recommendations: compare user_vector to mainstream cluster centroids, find the 3 features with largest gaps, rank by SHAP importance from M3. Output: 'Increase energy from 0.45 to the 0.65-0.75 range — this is the primary driver of mainstream playlist placement in your genre cluster.'
- For Path B recommendations: identify which underserved feature combination the user is closest to, suggest doubling down on it. Output: 'Your high acousticness + moderate valence profile sits in a low-competition space with 67 avg popularity. Lean into this.'
- For Path C recommendations: blend A and B suggestions with a 60/40 weight. Suggest testing both audiences.
- IMPORTANT: Every recommendation must include a confidence level (Low/Medium/High) based on how many similar songs exist in the training data.

- Save module as src/recommender.py.

Step 4: Difference Mapping Module (Day 8–9)

- This module powers the 'What makes you unique?' section of the UI.
- Function: map_differences(user_vector, similar_artist_vectors) -> for each feature, compute: delta, direction (above/below), and label.
- Labeling logic: delta > +0.2 AND feature is in top-5 SHAP importance = 'KEY DIFFERENTIATOR'. delta < -0.2 AND feature is in top-5 SHAP importance = 'GROWTH OPPORTUNITY'. Else = 'WITHIN RANGE'.
- Output example: [{feature: 'energy', user_value: 0.45, avg_reference: 0.72, delta: -0.27, label: 'GROWTH OPPORTUNITY', shap_rank: 2}].
- Save as src/difference_mapper.py.

Deliverables Checklist for M4

- /data/processed/market_gaps.csv
- /data/processed/feature_opportunities.json
- src/recommender.py — generate_recommendations() function
- src/difference_mapper.py — map_differences() function
- market_analysis.ipynb with charts for the presentation

MEMBER 5 — Frontend & UX Developer

MISSION: Build the UI that makes judges say 'I would actually use this.' A bad UI kills a great ML project. A great UI saves an average one.

Days 5–12. Use Streamlit. Do NOT attempt React unless you can finish in 3 days.

Why Streamlit (Not React)

Technology Decision: Streamlit vs React

Streamlit = Python, fast to build, works directly with your ML pipeline, no separate backend needed.
React = More beautiful but requires REST API setup, JSON serialization, CORS handling — adds 3 days of overhead.
For a competition demo with a 17-day window: Streamlit is the correct choice.
Streamlit can be made beautiful with custom CSS injection and careful layout. See Step 4.

Step 1: Set Up Streamlit App Structure (Day 5)

- Install: pip install streamlit plotly librosa
- Create app.py as the entry point. Use st.set_page_config(layout='wide', page_title='MusicGrowth Lab', page_icon='🎵').
- Create a multi-page structure using Streamlit's pages/ directory OR a sidebar radio selector with 5 pages: (1) Upload & Analyze, (2) Sound DNA Profile, (3) Market Position Map, (4) Growth Paths, (5) Recommendations.
- Add a loading state for every ML inference step using st.spinner().

Step 2: Screen 1 — Upload & Analyze (Day 6)

- File uploader: st.file_uploader('Upload your track', type=['mp3', 'wav']). On upload, call librosa to extract features.
- librosa extraction function (get this from M3 or build it): extract_features(file_path) -> returns dict matching Sound DNA vector format.
- Show a 'processing' animation while features are extracted.
- Display a success card with: Track Duration, Detected Tempo (BPM), Primary Genre Cluster (from M2's KMeans), Dominant Mood Descriptor (map valence + energy to mood: High energy + high valence = 'Euphoric', Low energy + low valence = 'Melancholic', etc.).

Step 3: Screen 2 — Sound DNA Profile (Day 7–8)

- RADAR CHART: use plotly.graph_objects.Scatterpolar. Show user's Sound DNA vector overlaid with the 3 most similar reference artists. This is the hero visualization of the entire product.
  - Features on radar: danceability, energy, valence, acousticness, instrumentalness, speechiness, liveness (7 axes). Normalize all to 0-1 scale.
- Difference table: below the radar, show M4's difference_mapper output as a color-coded table: green = KEY DIFFERENTIATOR, orange = GROWTH OPPORTUNITY, gray = WITHIN RANGE.
- Add a short text summary at the top generated by a simple template: 'Your sound is [X]% similar to [Artist 1] and [Y]% similar to [Artist 2]. You are [stronger/weaker] in energy and [stronger/weaker] in acousticness.'

Step 4: Screen 3 — Market Position Map (Day 8–9)

- UMAP SCATTER PLOT: use plotly.express.scatter. Plot all songs from embedding_2d.csv colored by cluster. Mark the user's position with a large star marker and a label 'YOU ARE HERE'.
- Add a slider/dropdown to overlay different metadata as color: genre, popularity, cluster name.

- Market Gap section below: plot M4's market_gaps.csv as a scatter with x=saturation_score, y=demand_score. Annotate clusters near the user's position. Title this chart: 'Where the Opportunities Are'.

Step 5: Screen 4 — Growth Path Simulator (Day 9–10)

- Show 3 cards side by side using st.columns(3): Path A (Mainstream), Path B (Niche), Path C (Hybrid).
- Each card shows: probability score from M3 (e.g., 'Mainstream Fit: 68%'), expected growth profile (use conditional templates based on probability ranges — do NOT hardcode fake numbers), top 3 SHAP features from M3, tradeoff summary from M4's recommendations.
- Add interactive sliders: 'What if you increase energy by 0.2?' — on slider change, re-run predict_paths() with modified vector and update the cards. THIS IS THE WOW MOMENT for judges.
- Show a simple bar chart comparing the 3 path probabilities.

Step 6: Screen 5 — Recommendations (Day 10–11)

- Show 3 collapsible sections (st.expander) — one per path.
- Each recommendation includes: specific feature adjustments with before/after values, confidence level badge (Low/Medium/High), and the market gap opportunity if applicable.
- Show the SHAP waterfall plot for the user's song (generated by M3) inline in this screen.

Step 7: Styling (Day 11–12)

- Inject custom CSS using st.markdown('<style>...css...</style>', unsafe_allow_html=True). Use dark theme: background #1a1a2e, accent purple #9b59b6, card backgrounds #16213e.
- Add the MusicGrowth Lab logo/title in the sidebar.
- Make sure every screen works with a sample pre-loaded song so the demo never fails (always have a fallback demo mode).

Deliverables Checklist for M5

- app.py — main Streamlit entry point
- pages/ or sidebar-based multi-screen navigation
- All 5 screens functional with real ML backend calls
- Custom CSS styling
- Demo mode with pre-loaded sample song
- All Plotly charts: radar, UMAP scatter, market gap, growth path bars, SHAP waterfall

MEMBER 6 — Integration Lead & Pitch Captain

MISSION: Make sure all 5 other members' work actually connects. You are the glue. You also own the pitch — judges vote based on how it's presented, not just how it's built.

Days 1–17. Active throughout. Delivery pressure intensifies from Day 10.

Step 1: Project Setup & Repository (Day 1)

- Create a GitHub repository: musicgrowth-lab (private). Invite all team members.
- Set up the folder structure: /data/raw/, /data/processed/, /models/, /src/, /notebooks/, /reports/, /app/
- Create a requirements.txt with: pandas, numpy, scikit-learn, xgboost, shap, umap-learn, librosa, plotly, streamlit, scipy, matplotlib, seaborn, pickle5.
- Write a SETUP.md explaining how to install dependencies and run the app. Every member must be able to run the full app locally.

Step 2: Daily Standups & Dependency Management (Day 1–9)

- Run a 15-minute daily standup (even on WhatsApp call). Every member answers: (1) What did I complete yesterday? (2) What am I doing today? (3) Any blockers?
- Maintain a shared task tracker (Google Sheet). Columns: Task, Owner, Status (Not Started / In Progress / Done / Blocked), Due Date.
- Track inter-member dependencies: M5 can't build inference screens until M1 has the scaler and M2 has the similarity engine. M3 can't compute SHAP until models are trained. Flag these immediately.

Step 3: Build the Inference Pipeline (Day 8–10)

- This is your primary technical contribution. Build src/pipeline.py — the single function that M5 calls.
- Function: run_full_analysis(audio_file_path) -> returns complete analysis dict.
- Pipeline steps inside the function: (1) Extract features via librosa. (2) Scale using loaded scaler.pkl. (3) Assign cluster via kmeans.pkl. (4) Find similar artists via similarity.py. (5) Map differences via difference_mapper.py. (6) Run 3-path predictions via predictor.py. (7) Generate recommendations via recommender.py. (8) Return everything as a single structured dict.
- Test with 3 different songs (different genres) to verify end-to-end. Log any errors immediately to the team.

Step 4: Model Evaluation Summary (Day 9–11)

- Collect all evaluation metrics from M3 and M4. Create a single /reports/model_eval_summary.md with a table of all metrics.
- Prepare backup static results for the demo: if live inference fails during judging, you need pre-computed results for 3 sample songs ready to display.

Step 5: Pitch Deck (Day 12–15)

- Slides structure (keep under 12 slides): (1) Problem — 10,000 songs released daily, 99.9% invisible. (2) Current Solutions — nothing quantitative exists for indie artists. (3) MusicGrowth Lab — the strategic positioning system. (4) How It Works — 4-step flow (Sound DNA → Similar Artists → Differences → 3 Paths). (5) Live Demo — hand off to M5 here. (6) Technical Architecture — data pipeline + ML stack. (7) Model Performance — key metrics (accuracy, F1, AUC). (8) Market Gap — opportunity visualization from M4. (9) Limitations & Future Work — be HONEST here, judges respect this. (10) Team & Timeline.
- Practice the Q&A with the full team. Prepare answers for every anticipated judge question from the original problem statement document.

Step 6: Final Integration Testing (Day 13–15)

- Full end-to-end test: fresh install of requirements.txt on a clean machine. Upload 3 different songs. Verify all 5 screens render correctly. Verify all models load without errors.
- Load testing: run the app for 30 minutes continuously. Fix any memory leaks or slow inference.

Step 7: Presentation Day Prep (Day 15–17)

- Prepare 2 demo tracks: one mainstream song (high energy, popular) and one niche/experimental song. These give contrasting and interesting outputs.
- Backup plan: export static screenshots of all screens for all 3 demo songs. If the app crashes during demo, show screenshots and explain what they show.
- Each team member should be able to answer questions about their specific component.

Deliverables Checklist for M6

- GitHub repo with clean structure and all code
- requirements.txt and SETUP.md
- src/pipeline.py — full end-to-end inference function
- /reports/model_eval_summary.md
- Pitch deck (Google Slides or PowerPoint)
- Demo songs prepared (2 contrasting tracks)
- Backup static screenshots

Part 4: Master Day-by-Day Timeline

Every task below has an owner. If you are blocked, tell M6 immediately. Do NOT wait.

DAY	TASK	OWNER	PRIORITY
Day 1	GitHub repo setup, folder structure, requirements.txt, SETUP.md	M6	CRITICAL
Day 1	Download Spotify Kaggle dataset (600K tracks), store in /data/raw/	M1	CRITICAL
Day 1	Run EDA notebook: null checks, distributions, class balance	M1	CRITICAL
Day 1	All members clone repo, install requirements, verify setup	ALL	HIGH
Day 2	Feature engineering: normalize, create Sound DNA vectors, popularity buckets	M1	CRITICAL
Day 2	Create mainstream_ready and niche_success target flags	M1	CRITICAL
Day 2	M5 begins Streamlit app skeleton: page structure, styling framework	M5	HIGH
Day 3	Export: spotify_processed.csv, sound_dna_matrix.npy, scaler.pkl	M1	CRITICAL
Day 3	Filter reference artists (1M+ followers), create reference_artists.csv	M1	CRITICAL
Day 3	M2 begins: load M1's matrix, run UMAP + PCA	M2	HIGH
Day 4	M2: K-Means clustering, silhouette score analysis, pick optimal k	M2	CRITICAL
Day 4	M3 begins: load data, understand target variables, set up XGBoost pipeline	M3	CRITICAL
Day 4	M4 begins: genre saturation analysis, create market_gaps.csv	M4	HIGH
Day 5	M2: cosine similarity engine, similarity.py module — tested and working	M2	CRITICAL
Day 5	M3: Path A mainstream classifier trained, metrics logged	M3	CRITICAL
Day 5	M5: Screen 1 (Upload & Analyze) — complete with librosa extraction	M5	HIGH
Day 6	M2: cluster_profiles.json and genre_space_viz.json exported	M2	HIGH
Day 6	M3: Path B niche classifier trained, metrics logged	M3	CRITICAL
Day 6	M4: feature combination analysis, feature_opportunities.json	M4	HIGH
Day 7	M3: Path C hybrid (multiclass) classifier trained, metrics logged	M3	CRITICAL
Day 7	M2: DBSCAN for outlier detection, update cluster labels	M2	MEDIUM
Day 7	M5: Screen 2 (Sound DNA / Radar Chart) — complete	M5	CRITICAL

Day 8	M3: SHAP explainability for all 3 models — explainer objects saved	**M3**	**CRITICAL**
Day 8	M4: recommender.py module — generate_recommendations() working	**M4**	**CRITICAL**
Day 8	M5: Screen 3 (Market Position Map / UMAP scatter) — complete	**M5**	**HIGH**
Day 8	M6 begins src/pipeline.py — wire M1+M2+M3+M4 into single function	**M6**	**CRITICAL**
Day 9	M3: predictor.py module — predict_paths() with SHAP explanations	**M3**	**CRITICAL**
Day 9	M4: difference_mapper.py module — map_differences() working	**M4**	**HIGH**
Day 9	M6: pipeline.py — end-to-end test with 3 sample songs	**M6**	**CRITICAL**
Day 10	ALL: full integration test — M5's app calls M6's pipeline successfully	**ALL**	**CRITICAL**
Day 10	M5: Screen 4 (Growth Path Simulator with sliders)	**M5**	**CRITICAL**
Day 10	FEATURE FREEZE — no new features after today	**M6**	**CRITICAL**
Day 11	M5: Screen 5 (Recommendations + SHAP waterfall)	**M5**	**HIGH**
Day 11	M3+M4: evaluation.ipynb complete with all metrics and charts	**M3+M4**	**HIGH**
Day 12	M5: full styling pass — CSS, dark theme, responsive layout	**M5**	**HIGH**
Day 12	M6: pitch deck draft — slides 1-10	**M6**	**HIGH**
Day 13	M6: model_eval_summary.md — collect all metrics from M3 and M4	**M6**	**HIGH**
Day 13	ALL: first full demo run-through — simulate judging session	**ALL**	**CRITICAL**
Day 14	Fix bugs surfaced in Day 13 demo	**ALL**	**CRITICAL**
Day 14	M6: prepare demo songs (2 contrasting tracks), verify outputs	**M6**	**HIGH**
Day 15	M5: final UI polish, demo mode with pre-loaded songs	**M5**	**HIGH**
Day 15	M6: pitch deck final, practice Q&A with full team	**M6**	**HIGH**
Day 16	Second full demo run-through — all 5 screens, timed	**ALL**	**CRITICAL**
Day 16	Prepare backup static screenshots for every screen	**M6**	**HIGH**
Day 17	Final polish, export backup screenshots, confirm all models load	**ALL**	**HIGH**
Day 17	Practice pitch ONE more time, brief every member on their Q&A answers	**M6**	**CRITICAL**

Part 5: Judge Q&A Preparation — Rehearse These

Every team member must know every answer below. Judges will try to find holes. These answers close those holes.

Q1: Isn't this just telling artists to copy successful songs?

Absolutely not — this is the opposite. We show where you're similar to successful artists, which gives you discoverability. We also show where you differ, which is your identity. We present three strategic paths — mainstream, niche, or hybrid — with honest tradeoffs. The artist chooses. We position; we don't prescribe.

Q2: How can audio features predict streaming success? There are so many other factors.

You're right, and we acknowledge that explicitly. Audio features alone don't predict virality — label support, social media, and timing all matter. We're not predicting absolute success. We're modeling conditional positioning: 'Songs with this audio profile historically appeared in X% more editorial playlists in this genre cluster.' These are positioning patterns, not causal guarantees.

Q3: Your R^2 or accuracy seems low. Your model isn't very good.

Our Path C classifier achieves X% F1-macro on popularity buckets. Yes, there's noise — because streaming success is inherently noisy. But notice: we're using SHAP to identify WHICH features matter most for EACH genre cluster. A globally low R^2 doesn't mean the directional insights are wrong. We care about the relative signal, not perfect prediction.

Q4: Won't all artists converge to the same sound if they all follow your recommendations?

This is exactly why we built the 3-path system. Path B (Niche) explicitly rewards uniqueness. We also highlight market gaps — underserved feature combinations with high demand and low supply. We're pointing artists AWAY from saturated mainstream clusters if their profile suits it. The system is designed to preserve diversity, not eliminate it.

Q5: How is this different from Spotify's own recommendation systems?

Spotify's systems are listener-facing — they connect existing songs with listeners. We're artist-facing and forward-looking — we analyze your unreleased or newly released music and tell you where it sits in the competitive landscape before it reaches listeners. Spotify ignores this problem entirely for independent artists.

Q6: Your dataset might be biased toward certain genres or time periods.

Valid concern. Our Kaggle dataset spans multiple genres and years. We trained separate models per path to reduce cross-genre bias. We also display confidence levels on each recommendation based on sample size — if a genre cluster has few training examples, the recommendation carries a 'Low Confidence' label. We don't pretend to have equal coverage everywhere.

Part 6: Complete Tech Stack & File Structure

LAYER	TECHNOLOGY	PURPOSE
Data	pandas, numpy	Data manipulation and feature matrix operations
Audio Extraction	librosa	Extract tempo, energy, spectral features from uploaded MP3/WAV files
Preprocessing	scikit-learn (StandardScaler)	Feature normalization, scaler persistence
Embedding / Reduction	UMAP, PCA (scikit-learn)	Dimensionality reduction for clustering and visualization
Clustering	K-Means, DBSCAN (scikit-learn)	Genre space segmentation, outlier detection
Similarity	cosine_similarity (scikit-learn)	Artist matching engine
ML Models	XGBoost + CalibratedClassifierCV	3-path growth prediction with calibrated probabilities
Explainability	SHAP (TreeExplainer)	Feature importance per prediction — the key differentiator
Visualization	Plotly Express / Graph Objects	Radar charts, UMAP scatter, bar charts, SHAP waterfall
Frontend	Streamlit	Full dashboard UI with 5 screens, file upload, interactive sliders
Version Control	Git + GitHub	Collaboration, code history, deployment

Project File Structure

Folder Structure (follow this exactly — deviations break the pipeline)

```
musicgrowth-lab/
  data/
    raw/          ← original Kaggle CSVs (never modified)
    processed/    ← all M1 outputs (CSV, NPY, JSON)
  models/         ← all .pkl files (scaler, KMeans, XGBoost, SHAP)
  src/
    similarity.py  ← M2's cosine similarity module
    predictor.py   ← M3's 3-path prediction + SHAP module
    recommender.py ← M4's recommendation generation module
    difference_mapper.py ← M4's feature delta module
    pipeline.py    ← M6's end-to-end inference function
  notebooks/       ← EDA, clustering, evaluation notebooks
  reports/         ← SHAP PNG charts, model_eval_summary.md
  app.py           ← M5's Streamlit entry point
```

```
requirements.txt
SETUP.md
README.md
```

Part 7: What Winning Actually Requires

MOST TEAMS WILL DO THIS (Losing Strategy)	YOU MUST DO THIS (Winning Strategy)
Build a classifier and show accuracy	Build 3 classifiers, explain WHY each feature matters using SHAP, show honest limitations
'Our model predicts hit songs with 80% accuracy'	'Our model identifies audio-based positioning clusters and conditional growth patterns with quantified uncertainty'
Generic Streamlit template with basic charts	Custom dark-themed UI, interactive sliders that re-run inference in real time, UMAP 'you are here' map
Avoid limitations section, fake confidence	State limitations explicitly — 'audio features alone do not capture marketing effects' — judges respect intellectual honesty
Everyone answers questions during Q&A, creating confusion	One person (M6) manages Q&A flow, other members speak only for their specific component when called upon

The Single Most Important Thing

Your demo must work flawlessly. A polished, smooth live demo that shows a real song being analyzed in real time
beats a technically superior project that crashes or shows errors. Spend Day 16 doing nothing but rehearsing the demo.
Always have the pre-computed backup screenshots. Never be caught empty-handed.

You caught the core flaw in the original concept and reframed it correctly. Now execute without cutting corners.

Good luck. Go win it. 🚀